

UnifAI - Server & Deployment Documentation

Audience: DevOps, IT administrators, SRE

Scope: requirements, environment, deployment (Docker compose / Helm), database, TLS, operations, upgrades, troubleshooting, hardening

Baseline: repo commit **16d4c3e**

1. What you are deploying

A single container/binary serves everything:

Served by the same process	Path/port
Admin + user dashboard (embedded React SPA)	/, /workspace/**
Management API	/api/**
AI inference API (OpenAI-compatible)	/v1/**
Provider pass-through APIs	/openai/**, /anthropic/**, /genai/**, /bedrock/**, ...
Browser AI guard control plane + agent ingestion	/api/browser-ai/**
WebSocket / realtime	/ws, /v1/realtime
Health	/health

Optional second container (compose profile **network-proxy**) runs the mitmproxy-based corporate proxy for office/GPO deployments.

2. Requirements

Component	Minimum	Recommended (production)
Container host	Linux with Docker 24+ / compose v2	Ubuntu 22.04+ / RHEL 9
CPU	2 vCPU	4-8 vCPU
RAM	2 GB	8-16 GB
Disk	10 GB	50 GB+ (logs + attachments + Guard packages)
Database	Postgres 13+	Postgres 15+ with TLS and automated backups
Outbound network	HTTPS to AI provider APIs	plus a corporate proxy if required
Ollama (only for AI Guard Bot rules)	reachable HTTP endpoint	dedicated CPU/GPU host
Reverse proxy	nginx / Caddy / HAProxy / Cloudflare	TLS 1.2+, HSTS
Build toolchain (only to build images)	Go 1.26+, Node 25+, Docker	-

Ports used by the stack (from **.env.example**):

Variable	Default	Purpose
APP_PORT	6000 (binary default 8001)	UnifAI HTTP / UI / API
PROXY_PORT	18182	Docker network proxy (profile network-proxy)
UNIFAI_PROXY_ADDR	127.0.0.1:18103	Laptop Guard local MITM bind
PAC_HTTP_PORT	18195	Laptop Guard local PAC/health server
MITM_WEB_PORT	18183	Dev-only mitmweb UI
UI_PORT	3100	Vite dev server (development only)

3. Environment variables

Copy **.env.example** -> **.env** and fill in real values. **.env** is git-ignored and should be the **only**

place live secrets exist.

3.1 Server / network

Variable	Required	Notes
SERVER_DOMAIN	yes	Public base URL, e.g. https://unifai.example.com . Drives CORS origins, MCP external URL, Guard config and PAC URLs. Compose fails fast if unset
APP_PORT	yes	Listening port
APP_HOST	no	Bind address (compose uses 0.0.0.0)
CONTAINER_NAME	yes	Container/hostname used for inter-container calls (raksha_tech)
PROXY_PORT	no	Docker network proxy port
DOCKER_NETWORK	no	Bridge network name (default unifai-network)
OLLAMA_DOCKER_NETWORK	no	External network containing the Ollama container (default 1panel-network)

3.2 Database

Variable	Required	Notes
DB_TYPE	yes	postgres (prod default), mysql or sqlite
DB_HOST , DB_PORT , DB_NAME , DB_USER , DB_PASSWORD	yes	Compose fails fast if unset
DB_SSL_MODE	no	disable locally; use require/verify-full in production

3.3 Auth, admin and crypto

Variable	Required	Notes
ADMIN_EMAIL, ADMIN_PASSWORD	yes	Bootstrap admin (change immediately after first login)
UNIFAI_ENCRYPTION_KEY	production	Long random secret; encrypts secrets stored in the DB
UNIFAI_GUARD_SECRET	production	Fleet secret shared by Guard agents (openssl rand -base64 32)
UNIFAI_GUARD_REQUIRE_SECRET	production (1)	Fail-closed when the Guard secret is missing/wrong
UNIFAI_PAC_ALLOW_QUERY_PROXY	no (0)	0 = PAC ignores a client ?proxy= (anti hijack)
PASSWORD_RESET_SECRET	yes	≥ 32 chars (openssl rand -base64 48); without it password reset returns HTTP 500
TRUST_PROXY_HEADERS	no (1)	Honour X-Forwarded-* only from trusted peers
TRUSTED_PROXIES	public edge	Comma-separated IPs/CIDRs of the edge proxy
CLUSTER_REPLICATE_SECRET	multi-node	Required by POST /internal/cluster/kv
ALLOW_LOCALHOST_CORS	no	1 allows credentialed CORS from localhost
ALLOW_OPEN_AUTH	no	1 allows unauthenticated admin while auth is disabled (local bootstrap only)

3.4 Logging, data and branding

Variable	Notes
LOG_LEVEL	debug / info / warn / error (default info)
LOG_STYLE	json (default) or pretty
APP_DIR	Data directory holding config.json , logs/ , pdf/ , attachments/ (default /app/data)
UNIFAI_COMPANY_NAME, UNIFAI_COMPANY_LOGO	Dashboard branding
OLLAMA_URL / BROWSER_AI_OLLAMA_URL	Ollama endpoint used by AI Guard Bot rules
UNIFAI_BACKEND_URL	Backend URL for the UI dev server and Guard

3.5 Guard agent (client side)

Variable	Notes
UNIFAI_BACKEND_URL / SERVER_DOMAIN	Backend the agent reports to
UNIFAI_PROXY_ADDR	Local MITM bind (127.0.0.1:18103)
UNIFAI_PAC_URL	PAC endpoint (defaults to <backend>/api/browser-ai/pac)
UNIFAI_PAC_SYNC_SECONDS	PAC/rule refresh interval (default 3, clamped 2-600)
UNIFAI_SERVER_MODE	1 for network/server proxy mode
UNIFAI_AGENT_TYPE	endpoint or network

Variable	Notes
UNIFAI_LISTEN_HOST	127.0.0.1 (endpoint) or 0.0.0.0 (network)
UNIFAI_PAC_ADVERTISE_ADDR	Address advertised in PAC for network mode
UNIFAI_AGENT_ID, UNIFAI_AGENT_HOSTNAME	Fleet identity
UNIFAI_NETWORK_AGENT_REGISTER	1 lets a network proxy register itself in the fleet
UNIFAI_FAIL_OPEN	1 = allow traffic if the backend is unreachable (use with care)
UNIFAI_EVAL_TIMEOUT	AI Guard Bot evaluation timeout in seconds

After changing **.env**, regenerate the Guard config:

```
python apps/browser-guard/scripts/sync_config_from_env.py
```

4. config.json reference

The runtime config is **APP_DIR/config.json** (**/app/data/config.json** inside the container). On first start the entrypoint copies **/app/configs/config.json** there. It supports **env.NAME** indirection so secrets are not stored in the file.

Structure observed in **configs/config.json**:

Key	Purpose
config_store	Config DB: enabled , type (postgres/mysql/sqlite), config .{ host , port , db_name , user , password , ssl_mode } (all via env.DB_*)
logs_store	Log DB - same shape as config_store
client	drop_excess_requests , initial_pool_size (5000), enable_logging , disable_content_logging , log_retention_days (365), enforce_auth_on_inference , allowed_origins (env.SERVER_DOMAIN), max_request_body_size_mb (100), mcp_external_client_url , compat.should_convert_params , disable_db_pings_in_health
governance	auth_config.is_enabled , auth_config.admin_username (env.ADMIN_EMAIL), admin_password (env.ADMIN_PASSWORD)
framework	pricing , plus framework-level feature configuration
plugins	Per-plugin config and enable/disable + ordering
providers	Provider definitions, keys and network config
mcp , vector_store , oauth2 , etc.	Optional subsystems

Also under **configs/**: **mcp-library.json** (MCP catalog seed), **model-parameters.json**, **pricing.json**.

Config is loaded into memory at boot; changes made through the UI are persisted to the DB and to

config.json. Do not bind-mount **config.json** as a single file - mount the whole **./data**

directory, otherwise atomic renames fail ("device or resource busy"). The entrypoint explicitly

errors out if **config.json** is not writable.

5. Docker compose deployment (recommended path)

5.1 Prepare

```
git clone <repo> && cd unifai_project cp .env.example .env # edit .env: SERVER_DOMAIN, DB_*, ADMIN_*,
UNIFAI_ENCRYPTION_KEY, # UNIFAI_GUARD_SECRET, PASSWORD_RESET_SECRET
```

5.2 Build and start

```
docker compose up -d --build # builds the UI + Go binary from source docker compose logs -f raksha_tech #
watch startup (plugin status table, listener line)
```

The build is multi-stage (**deploy/docker/Dockerfile.local**):

node:25-alpine builds the React UI -> **golang:1.26.4-alpine** builds the binary with a Go workspace

-> **alpine:3.23** runtime with **musl libgcc ca-certificates zlib git**, non-root **appuser**, a healthcheck on **/health**, and the Guard release artifacts copied to **/app/release**.

5.3 Volumes and mounts (from **docker-compose.yml**)

Mount	Purpose
./data:/app/data	Runtime state: config.json , logs, attachments, PDFs
./configs:/app/configs	Default configs
./apps/browser-guard/release:/app/release	Guard install packages served by Setup downloads
./apps/browser-guard/proxy:/app/guard-proxy:ro	Guard proxy source for Rebuild & Publish
./apps/browser-guard/agent:/app/guard-agent:ro	Guard agent source for Rebuild & Publish

extra_hosts: host.docker.internal:host-gateway lets the container reach host services (e.g. Ollama);

DNS is pinned to **8.8.8.8 / 1.1.1.1**; the container joins both the default bridge network and the external Ollama network (**1panel-network** by default - create it once if it does not exist).

5.4 Optional network proxy (office / GPO)

```
docker compose --profile network-proxy up -d raksha_browser_ai_proxy
```

This runs **mitmproxy/mitmproxy:latest** with **mitmdump -s /app/proxy/browser_ai_proxy.py -p PROXY_PORT**

and environment **UNIFAI_BACKEND_URL=http://raksha_tech:APP_PORT**, **UNIFAI_SERVER_MODE=1**,

UNIFAI_AGENT_TYPE=network, UNIFAI_FAIL_OPEN=0, UNIFAI_GUARD_SECRET=...

Point office PAC/GPO to:

```
https://<SERVER_DOMAIN>/api/browser-ai/pac?proxy=<proxy-host>:18182
```

and distribute the mitmproxy CA to client machines for HTTPS inspection.

Never MITM the same browser session with both the laptop Guard and the corporate proxy.

5.5 Verify

```
curl -fsS https://<SERVER_DOMAIN>/health curl -fsS https://<SERVER_DOMAIN>/api/browser-ai/proxy.pac curl
-fsS https://<SERVER_DOMAIN>/api/browser-ai/setup/info
```

Expect JSON (or a PAC script), never the HTML login page - HTML means the backend build does not include the Browser AI routes.

6. Database setup and migrations

6.1 Postgres (production)

```
CREATE DATABASE unifai_new; CREATE USER agent_unify WITH ENCRYPTED PASSWORD '<strong-password>'; GRANT ALL
PRIVILEGES ON DATABASE unifai_new TO agent_unify; \c unifai_new GRANT ALL ON SCHEMA public TO agent_unify;
```

Then set **DB_TYPE=postgres, DB_HOST, DB_PORT=5432, DB_NAME, DB_USER, DB_PASSWORD, DB_SSL_MODE=require.**

6.2 Automatic schema management

- On boot the server connects via GORM, creates/updates tables and seeds defaults: RBAC catalog, three system roles, the single-row Browser AI control settings, and any missing config rows.
- framework/migrator (addcolumn.go, dropcolumn.go)** and **configstore/migrations.go / logstore/migrations.go** handle additive schema changes. Migrations are safe to run repeatedly.
- framework/configstore/tables/migrations** run for both the config DB and the log DB.

6.3 Tables you will care about in operations

Area	Tables
Identity	governance_users, sessions, rbac_roles, access_profiles, audit_logs
Governance	virtual keys + their link tables, teams, customers, business units, budgets, rate limits
Provider config	provider rows + keys, models, model configs, pricing, pricing overrides
Logs	LLM log tables, mcp_logs , dashboard materialized views

Area	Tables
Browser AI	browser_ai_logs , browser_ai_search_logs , browser_guard_rules , browser_ai_target_websites , browser_ai_control_settings , browser_ai_agents , browser_ai_agent_settings , browser_guard_fleet_config , browser_guard_rebuild_logs , browser_ai_warning_emails
Ops	browser_guard_rebuild_logs , distributed-lock rows, feature flags, plugins

6.4 Retention

client.log_retention_days (default 365) drives the log-retention cleaner started during bootstrap;

0 disables the cleaner. Async inference jobs are cleaned by the async-job cleaner.

Browser AI attachments are disk files under **APP_DIR/attachments** with DB expiry metadata - the retention job and manual deletes clean them.

7. Reverse proxy and TLS

Terminate TLS at the edge; the container serves plain HTTP internally.

nginx example

```
server { listen 443 ssl http2; server_name unifai.example.com; ssl_certificate /etc/ssl/fullchain.pem;
ssl_certificate_key /etc/ssl/privkey.pem; client_max_body_size 100m; # match client.max_request_body_size_mb
location / { proxy_pass http://127.0.0.1:6000; proxy_http_version 1.1; proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme; # WebSocket / SSE proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade"; proxy_read_timeout 3600s; proxy_buffering off; # required for
streaming responses } }
```

Important:

- Set **TRUST_PROXY_HEADERS=1** and **TRUSTED_PROXIES=<edge IP/CIDR>** so **X-Forwarded-*** is honoured (this controls the cookie **Secure** flag and client IP for rate limits/logs).
- Disable proxy buffering for **/v1/*** so SSE streaming is not buffered.
- Cloudflare or any CDN counts as a proxy - list its egress ranges in **TRUSTED_PROXIES**.

8. Kubernetes / Helm

Chart: **deploy/helm/unifai** (**Chart.yaml**, **values.yaml**, **values.schema.json**, templates for deployment, service, ingress, HPA, secrets, configmap, service account/RBAC, and optional backing services: Postgres, Redis, Qdrant, Weaviate).

```
cd deploy/helm ./scripts/generate-values.sh # write a values file for your environment ./scripts/validate.sh
# schema/sanity validation ./scripts/install.sh # helm install # or manually: helm install unifai ./unifai -f
my-values.yaml
```

Values examples provided: **sqlite-only**, **sqlite-redis**, **sqlite-qdrant**, **sqlite-weaviate**,

postgres-only, **postgres-redis**, **postgres-qdrant**, **postgres-weaviate**, **mixed-backend**, **external-postgres**, **production-ha**, **providers-and-virtual-keys**, **secrets-from-k8s**.

For production HA:

- Use **production-ha.yaml** as a starting point (replicas + HPA + external Postgres).
- Set **CLUSTER_REPLICATE_SECRET** so nodes can replicate config through **POST /internal/cluster/kv**.
- Prefer a managed/external Postgres with TLS and backups over the in-chart Postgres.
- Set an Ingress with TLS and **proxy-body-size** ≥ 100 MB; keep SSE working (disable buffering).

Repackage/regenerate the chart index after value/template changes: **make helm-index**.

9. Operations

9.1 Start, stop, restart

```
docker compose up -d --build # build + start
docker compose restart raksha_tech # restart app only
docker compose stop # stop all
docker compose down # remove containers (data stays in ./data and the DB)
docker compose ps # status
```

Graceful shutdown: the binary handles SIGINT/SIGTERM, closes realtime sessions, shuts down the core client and storage engines with a 30 s timeout, and also drains the pprof server (90 s).

9.2 Logs

- Container logs: **docker compose logs -f --tail=200 raksha_tech** (JSON by default; **LOG_STYLE=pretty** for humans, **LOG_LEVEL=debug** when diagnosing).
- Startup line to look for: **successfully started unifai, serving UI on http://0.0.0.0:<port>**, plus the plugin status table (**plugin status: <name> - <status>**).
- App file logs under **APP_DIR/logs/**.
- Request/business logs are in the dashboard (LLM Logs, MCP Logs, Browser AI -> Prompt Logs) and in the log database - not only in stdout.

9.3 Health and monitoring

- Liveness/readiness: **GET /health** -> 200 **{"status":"ok", ...}**; 503 lists the failing component.

The image healthcheck already calls this every 30 s.

- Watch: CPU/RAM of the container, DB connections, log-table growth, **browser_ai_logs** growth, Guard agent heartbeats (Browser AI -> Guard Agents -> last seen), rebuild history.
- Enable OpenTelemetry (**otel** plugin) and connectors (Datadog/New Relic/BigQuery/Kafka/PubSub) for external observability. Alert channels deliver webhook notifications.
- Dev/diagnostic: pprof endpoints exist only in dev builds; **/pprof** page in the UI.

9.4 Backups and restore

Data	How to back up
Postgres (ConfigStore + LogsStore)	pg_dump / managed snapshots - this is the critical asset
./data	archive the directory (config.json , attachments, PDFs, logs)
./configs	keep under version control
Guard packages	apps/browser-guard/release/ (rebuildable, but keep for consistency)
.env	store in a secret manager; never in git

Restore order: database -> **./data** -> recreate the container with the same **.env**.

Recommended: nightly **pg_dump** with 30-day retention, weekly full **./data** archive, and an off-site copy of **.env/secrets**.

9.5 Upgrade procedure

1. Back up the DB and **./data**.
2. **git pull** (and **make sync-guard-config** if Guard settings changed).
3. Rebuild and restart: **docker compose up -d --build**.
4. Watch startup logs; schema migrations and RBAC seeding run automatically.
5. Verify **GET /health**, log in, open Browser AI (targets, rules, agents), and send a test prompt through a provider.
6. Rollback: check out the previous commit, rebuild, restore the DB dump if a schema change is incompatible.

Perf-critical rebuild: the Docker build compiles the UI (Node) then the Go binary - expect several minutes on first build. Docker layer cache makes subsequent builds much faster.

9.6 Scaling and HA

- The app is stateless apart from the DB and **APP_DATA** - run multiple replicas behind a load balancer.
- Multi-node config replication needs **CLUSTER_REPLICATE_SECRET**; see Workspace -> Cluster.
- Postgres is the shared bottleneck: use a managed instance, connection pooling, and monitor slow queries.
- Use the adaptive/load-balancer routing (**/api/load-balancer**) and circuit breaker policies to spread traffic across providers/keys and isolate failing ones.
- Guard agents heartbeat to any replica (the fleet is stored in the DB).

9.7 Guard fleet operations (server side)

- **Setup -> Download Setup ZIP** (**/api/browser-ai/setup/download[-windows|-mac].zip**) serves the packages in **apps/browser-guard/release/**.
- **Setup -> Rebuild & Publish** (**POST /api/browser-ai/setup/rebuild**) republishes Guard Python code as a bundle; history is in **/api/browser-ai/setup/rebuild-history** and **browser_guard_rebuild_logs**.
- **Uninstall key**: **GET|PUT /api/browser-ai/agents/uninstall-key** (company) or per-agent rotate.
- **Remote uninstall**: **POST /api/browser-ai/agents/{id}/remote-uninstall**.
- **Fleet config**: **GET|PUT /api/browser-ai/fleet-config** - agents pull it on heartbeat.

- When you rotate **UNIFAI_GUARD_SECRET**, re-sync Guard config and re-publish packages; existing agents keep working only if their config is updated.

10. Troubleshooting

Symptom	Likely cause	Fix
Compose refuses to start: set SERVER_DOMAIN / DB_HOST / ADMIN_EMAIL ... in .env	Required variable missing	Fill the variable in .env - compose uses fail-fast substitution
Error: /app/data/config.json is not writable	config.json mounted as a single file	Mount only ./data:/app/data
/api/browser-ai/* returns the login page HTML	Image lacks Browser AI routes, or an edge rule intercepts the path	Deploy the current backend; exclude /api/browser-ai/* from SSO redirects and SPA fallback
/health returns 503 config store not available	Wrong DB credentials/host, DB down, TLS mismatch	Check DB_* , DB_SSL_MODE , and DB reachability from the container
Agents never appear under Guard Agents	Wrong backend URL in Guard config, missing/mismatched UNIFAI_GUARD_SECRET , egress blocked	Re-run sync_config_from_env.py , verify /api/browser-ai/agents/heartbeat returns JSON, check firewall/proxy
Browsers show certificate warnings on AI sites	MITM CA not trusted on the endpoint	Check ca_install_status.txt ; re-trust the CA; verify GPO/MDM distribution
Prompts are not appearing in Prompt Logs	Target domain not monitored, PAC not applied, agent not running	Add/verify the Target Website, confirm PAC is applied, check unifai_guard.log
Rules do not take effect immediately	Rule cache (~2 s) + PAC sync interval (default 3 s)	Wait a few seconds or lower UNIFAI_PAC_SYNC_SECONDS (min 2)
Password reset returns 500	PASSWORD_RESET_SECRET missing or < 32 chars	Set a ≥ 32-char secret and restart
Login blocked: "Too many attempts..."	Per-IP (30/15 min) or per-username lockout	Wait for the window or clear the lockout rows for that key
Streaming responses arrive all at once	Reverse-proxy buffering	proxy_buffering off (nginx) / disable buffering
Cookie not marked Secure behind HTTPS	TRUST_PROXY_HEADERS off, or edge not listed	Set both so X-Forwarded-Proto: https is trusted
AI Guard Bot rules never evaluate	Ollama/LLM unreachable from backend or agent	Verify OLLAMA_URL / BROWSER_AI_OLLAMA_URL and /api/browser-ai/ollama-models
Duplicate blocks / duplicate log rows	Laptop Guard and corporate proxy both MITM the session	Use one path per network location
Container OOM or slow	Log tables + attachments grew, or heavy model-catalog fetch	Check retention and disk, raise memory, add DB pooling
MCP clients fail then recover	Expected: the health monitor reconnects	Confirm the MCP server is reachable; check MCP client status in the UI

11. Security hardening checklist

- [] Long random **UNIFAI_ENCRYPTION_KEY**, **UNIFAI_GUARD_SECRET**, **PASSWORD_RESET_SECRET**, **CLUSTER_REPLICATE_SECRET** - generated with **openssl rand -base64 32|48**, stored in a secret manager.
- [] **ADMIN_PASSWORD** changed after first login; bootstrap admin replaced by a named admin account.
- [] **UNIFAI_GUARD_REQUIRE_SECRET=1** and **UNIFAI_PAC_ALLOW_QUERY_PROXY=0**.
- [] **DB_SSL_MODE=require/verify-full**; database not publicly exposed; least-privilege DB user.
- [] TLS at the edge, HSTS, HTTP->HTTPS redirect; **TRUST_PROXY_HEADERS=1** with an explicit **TRUSTED_PROXIES** list.
- [] **client.allowed_origins** limited to your domain (no wildcard); **ALLOW_LOCALHOST_CORS** unset.
- [] **enforce_auth_on_inference** enabled; provider keys stored through encrypted governance config, never in source or compose.
- [] Least-privilege RBAC: prefer **sub_admin**/custom roles over **admin**; review Governance -> Roles & Permissions and Access Profiles periodically.
- [] Registration approval enabled for sign-ups.
- [] SMTP configured so password reset and warning e-mails work (**/api/smtp-config**).
- [] Audit Logs reviewed regularly; alert channels configured for critical events.
- [] Backups automated and **restore-tested**; **.env** and DB dumps encrypted at rest.
- [] Guard uninstall key set, rotated periodically, shared only with IT.
- [] Guard CA distribution via GPO/MDM; macOS MDM profile **apps/browser-guard/installer/mdm/disable_private_relay.mobileconfig** applied.
- [] Guard agent heartbeats monitored; investigate agents missing for more than 7 days.
- [] Rotate **UNIFAI_GUARD_SECRET** and uninstall keys on staff/IT changes.
- [] Keep the container image, Postgres and the host patched; review **Dockerfile** base-image digests.

12. Quick reference

Make targets (build/deploy)

```
make build build UI + Go binary make docker-image build image (LOCAL=1 → Dockerfile.local) make docker-run
run the container make helm-index repackage the Helm chart + regenerate index.yaml make sync-guard-config
regenerate Guard config/docs from .env make build-guard-windows Windows Guard installer + ZIP make
build-guard-mac macOS Guard app + ZIP (on a Mac) make build-guard-mac-pkg macOS .pkg (on a Mac) make docs
bundle OpenAPI + Mintlify docs dev server
```

Key files

File	Why it matters
.env / .env.example	All configuration and secrets
docker-compose.yml	Container, volumes, networks, network-proxy profile
deploy/docker/Dockerfile.local	Production image build
deploy/docker/docker-entrypoint.sh	Volume permissions, config bootstrap, argument handling

File	Why it matters
configs/config.json	Runtime config template
apps/browser-guard/config/unifai_guard_config.json	Generated Guard agent config
apps/browser-guard/release/	Guard installers served to employees
apps/browser-guard/release/VERSION.txt	Guard build version

Verification endpoints

```
GET /health GET /api/version GET /api/browser-ai/proxy.pac GET /api/browser-ai/pac?proxy=<host:port> GET  
/api/browser-ai/targets GET /api/browser-ai/rules GET /api/browser-ai/fleet-config GET  
/api/browser-ai/setup/info GET /api/browser-ai/agents
```